

# Anteraja Tracking & Operations

PostgreSQL relational database untuk tracking paket, operasional seller, settlement, dan rekonsiliasi mutasi bank.

**Evan William**

Branch 6-db

# 15

tabel publik

# 3NF

normalisasi inti

# RLS

isolasi per akun

# Kebutuhan tugas sudah dipetakan ke artefak

---

## Relational database

PostgreSQL melalui Supabase dengan PK, FK, composite FK, constraint, index, trigger, RLS, dan RPC.

## Analisis FRD dan UI

FRD\_DATA\_MAPPING.md menautkan AUTH, TRK, OPS, FIN, dan komponen UI ke tabel atau function.

## Tabel pendukung

15 tabel publik mencakup tracking, tindakan penerima, tiket, notifikasi, seller, Finance, dan settlement.

## ERD .webp

Diagram final menampilkan domain, atribut kunci, dan relasi utama dalam satu kanvas.

## Sample data

Satu akun demo, lima shipment utama, timeline, settlement, impor bank, tiket, preferensi, dan outbox.

## Dokumentasi

Run guide, kamus data, normalisasi, ERD Mermaid, query verifikasi, dan smoke test tersedia.

# Satu sumber data untuk tiga area produk

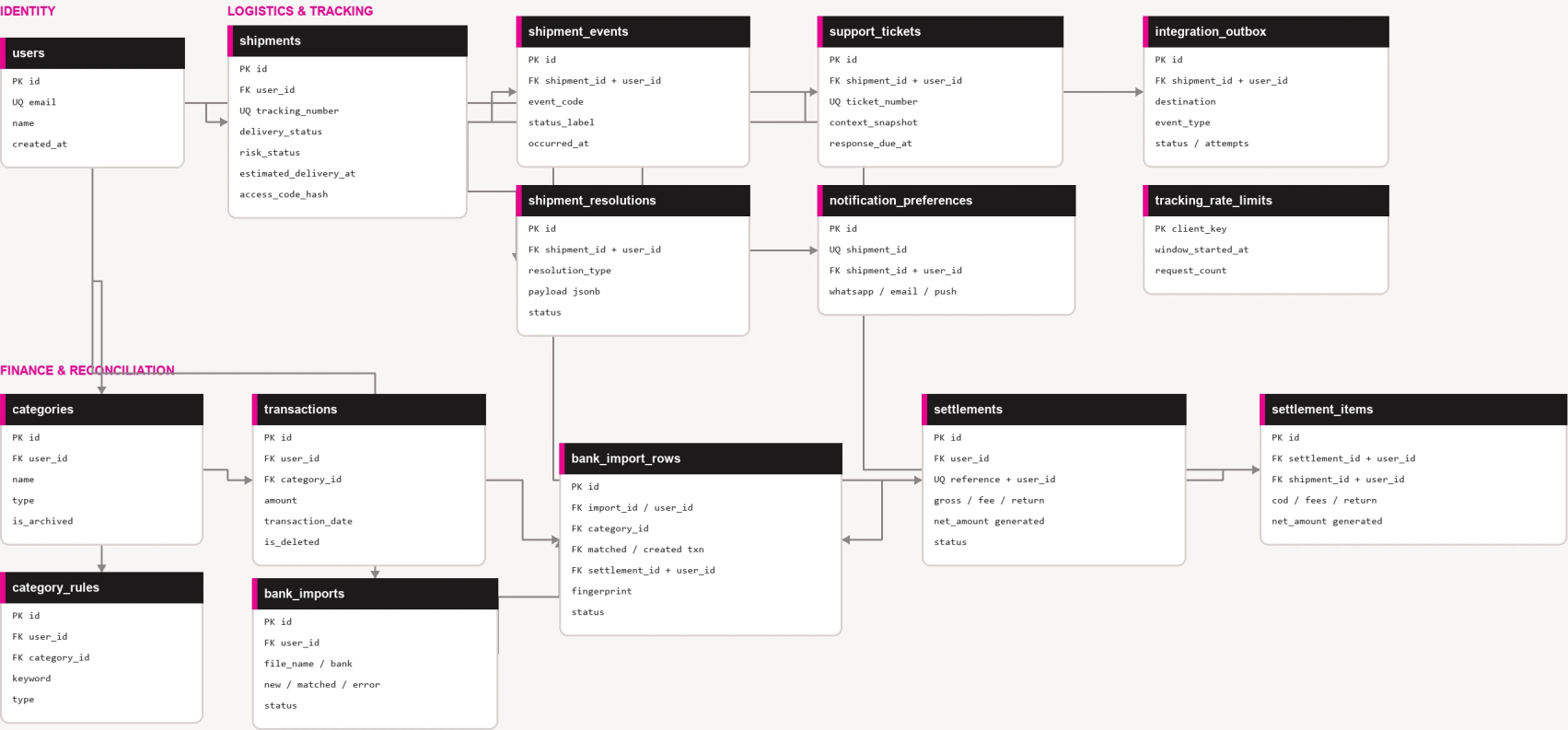
| TRACKING                 | SELLER OPERATIONS  | FINANCE               |
|--------------------------|--------------------|-----------------------|
| <b>shipments</b>         | <b>shipments</b>   | <b>transactions</b>   |
| shipment_events          | risk_status        | categories dan rules  |
| shipment_resolutions     | search dan filter  | bank imports dan rows |
| support_tickets          | RLS per user       | settlements           |
| notification_preferences | integration_outbox | settlement_items      |

shipment\_id menjadi jembatan antara perjalanan paket dan pencairan COD. user\_id menjaga seluruh relasi tetap berada pada akun yang sama.

# Relasi utama dan atribut kunci

## ERD - Anteraja Tracking & Operations

PostgreSQL / Supabase | 15 tabel publik | PK, FK, composite FK, RLS



Notasi: PK = primary key | FK = foreign key | UQ = unique | panah menuju tabel child  
Composite FK menyatakan user\_id untuk mencegah relasi lintas akun.

# Skema lengkap dapat diperiksa dalam satu file

---

## 00\_full\_schema.sql

database/00\_full\_schema.sql

```
1  -- ANTERAJA TRACKING & OPERATIONS - FULL POSTGRESQL SCHEMA
2  -- Generated from supabase/migrations in timestamp order.
3  -- Run once on an empty Supabase project. Do not run after migrations.
4
5
6  -- =====
7  -- SOURCE: supabase/migrations/20260101000000_create_users.sql
8  -- =====
9
10 -- Tabel users: cerminan auth.users yang bisa dibaca dari aplikasi.
11
12 create table if not exists public.users (
13   id uuid primary key references auth.users (id) on delete cascade,
14   email text not null unique,
15   name text,
16   created_at timestampz not null default now()
17 );
18
19 -- Email selalu disimpan lowercase.
20 alter table public.users
21   add constraint users_email_lowercase check (email = lower(email));
22
23 alter table public.users enable row level security;
24
25 create policy "users_select_own" on public.users
26   for select using (auth.uid() = id);
27
```

### Sumber deployment

File migration tetap menjadi sumber deployment. 00\_full\_schema.sql menggabungkannya agar proses penilaian lebih mudah.

# Data contoh mencakup alur normal dan kendala

## 01\_sample\_data.sql

database/01\_sample\_data.sql

```
93 insert into public.users (id, email, name)
94 values ('11111111-1111-4111-8111-111111111111', 'demo@contoh.test', 'Peserta Demo')
95 on conflict (id) do nothing;
96
97 -----
98 -- Kategori
99 -----
100
101 insert into public.categories (id, user_id, name, type, is_archived) values
102 ('22222222-0000-4000-8000-000000000001', '11111111-1111-4111-8111-111111111111', 'Biaya pengiriman', 'expense', false)
103 ('22222222-0000-4000-8000-000000000002', '11111111-1111-4111-8111-111111111111', 'Biaya layanan', 'expense', false)
104 ('22222222-0000-4000-8000-000000000003', '11111111-1111-4111-8111-111111111111', 'Retur dan klaim', 'expense', false)
105 ('22222222-0000-4000-8000-000000000004', '11111111-1111-4111-8111-111111111111', 'Biaya bank', 'expense', false)
106 ('22222222-0000-4000-8000-000000000005', '11111111-1111-4111-8111-111111111111', 'Operasional hub', 'expense', false)
107 ('22222222-0000-4000-8000-000000000006', '11111111-1111-4111-8111-111111111111', 'Kategori lama', 'expense', true),
108 ('22222222-0000-4000-8000-000000000007', '11111111-1111-4111-8111-111111111111', 'Settlement COD', 'income', false)
109 ('22222222-0000-4000-8000-000000000008', '11111111-1111-4111-8111-111111111111', 'Koreksi settlement', 'income', false)
110
111 -----
112 -- Transaksi (tersebar di ~3 bulan terakhir, relatif terhadap hari ini)
113 -----
114
115 insert into public.transactions (user_id, category_id, amount, description, transaction_date)
116 select
117 '11111111-1111-4111-8111-111111111111',
118 ('22222222-0000-4000-8000-00000000000' || seed.category)::uuid,
119 seed.amount,
```

Akun demo

demo@contoh.test | resi ANT-100015 | kode akses 260926. Seed juga memuat settlement, impor bank, tiket, preferensi, dan outbox.

# Setup dan verifikasi dapat diikuti dari awal

---

## Menjalankan Database Anteraja Tracking & Operations

Panduan ini menyiapkan database PostgreSQL melalui Supabase CLI, memuat sample data, dan memverifikasi bahwa tabel, relasi, RLS, serta RPC tersedia. Gunakan database lokal atau project development karena perintah reset menghapus data yang ada.

### Prasyarat

- Node.js sesuai versi pada project.
- Docker Desktop aktif untuk Supabase lokal.
- Supabase CLI tersedia melalui `npx`.
- File `.env.local` hanya berada di komputer Anda dan tidak masuk Git.

### Opsi A: jalankan dengan Supabase lokal

1. Buka terminal di root repository.
2. Instal dependency aplikasi:

```
```bash
npm install
```
```

3. Mulai layanan Supabase lokal:

```
```bash
npx supabase start
```
```

4. Terapkan seluruh migration dan sample data:

```
```bash
npx supabase db reset
```
```

### Panduan utama

database/DATABASE\_RUN.md menjelaskan setup lokal, SQL Editor, smoke test, alur verifikasi, integritas, dan troubleshooting.

# Struktur 3NF dengan denormalisasi yang terukur

---

- **1NF - nilai atomik**

Timeline dan settlement item menjadi baris terpisah, bukan array di tabel induk.

- **2NF - bergantung pada relasi**

Komponen COD dan fee berada pada pasangan settlement-shipment.

- **3NF - master tidak diulang**

Kategori, header impor, detail impor, header settlement, dan profil user dipisah.

- **Composite FK**

shipment\_id atau settlement\_id selalu dipasangkan dengan user\_id pada relasi kritis.

- **Constraint dan generated column**

Status, nominal, koordinat, panjang input, dan nilai bersih divalidasi oleh PostgreSQL.

- **Snapshot yang disengaja**

Tiket CS dan outbox menyimpan JSON immutable untuk audit dan retry.

# Branch dan paket siap diperiksa

---

## Branch

# 6-db

## Link repository

<https://github.com/evan-william/anteraja-capstone-evanwilliam/tree/6-db>

## Isi branch

File SQL, sample data, ERD .webp, kamus data, pemetaan FRD, penjelasan normalisasi, run guide, query verifikasi, dan PDF pengumpulan.

Berkas LMS: day6-database-report.pdf